Labo1 Geavanceerde Computer Architecture

Arne Van der Stede en Syme Vandenbosch

Academiejaar 2025-2026
February 19, 2026

1 Introduction

Voor deze opdracht bestaat de opgave eruit om een array om te kunnen keren (Laatste waarde wordt de eerste waarde en vica versa). Om op deze manier een eerste ervaring te hebben met de werking in de programmeer omgeving en vertrouwd te geraken met de Cuda syntax.

2 Code

```
#include <iostream>
#include <cuda_runtime.h>

__global__ void flip(const int *in, int *out, int N){
    int idx = threadIdx.x + blockIdx.x * blockDim.x; // blockDim --> #threads per block
    if (idx < N) {
        out[idx] = in[N-1-idx];
    }
}

int main() {
    const int N = 9;
    int arr_in[N], arr_out[N];
    int *d_in, *d_out;

    for(int i = 0; i < N;i++){
        arr_in[i] = i;
    }

    //GPU memory allocation
    cudaMalloc((void**)&d_in, N * sizeof(int));
    cudaMalloc((void**)&d_out, N * sizeof(int));

    //Memory miration
    cudaMemcpy(d_in, arr_in, N * sizeof(int), cudaMemcpyHostToDevice);

    //timings
    cudaEvent_t start_cuda, stop_cuda;
    cudaEventCreate(&start_cuda);
    cudaEventCreate(&stop_cuda);

    cudaEventRecord(start_cuda);
    // Launch kernel (1 block of N threads)
    flip<<<3, 3>>>(d_in, d_out, N);

    cudaDeviceSynchronize();
    cudaEventRecord(stop_cuda);
    cudaEventSynchronize(stop_cuda);

    float ms;
    cudaEventElapsedTime(&ms, start_cuda, stop_cuda);

    // Copy result back to host
    cudaMemcpy(arr_out, d_out, N * sizeof(int), cudaMemcpyDeviceToHost);
```

```
45
46      std::cout << "Input: ";
47      for (int i = 0; i < N; i++) {
48          std::cout << arr_in[i] << " ";
49      }
50      std::cout << std::endl;
51
52      std::cout << "Result: ";
53      for (int i = 0; i < N; i++) {
54          std::cout << arr_out[i] << " ";
55      }
56      std::cout << std::endl;
57      std::cout << "Elapsed Time: ";
58      std::cout << ms << " " ;
59      std::cout << "ms" << std::endl;
60
61      // Free device memory
62      cudaFree(d_in);
63      cudaFree(d_out);
64      cudaEventDestroy(start_cuda);
65      cudaEventDestroy(stop_cuda);
66
67      return 0;
68  }
```

Listing 1: code flip

3 Observaties

3.0.1 Open vragen

OQ1 — Hoe bepaal je de block- en gridgrootte (aantal threads per block en aantal blocks in de grid)?
De block- en gridgrootte wordt gekozen in functie van de grootte van N en de en de beschikbare GPU-architectuur. Het totaal aantal threads moet minstens gelijk zijn aan het aantal elementen.

OQ2 — Hoe weet een thread welk element ze moet verwerken?
Elke thread berekent zelf de globale index met ingebouwde CUDA-variabelen. In 1D gebeurt dit door gebruik te maken van volgende functie:

$$idx = threadIdx.x + blockIdx.x * blockDim.x.$$

Hierbij is threadIdx.x de threadpositie binnen het blok, blockIdx.x het bloknummer en blockDim.x het aantal threads per blok. Deze formule geeft elke thread een unieke index. Die gebruikt kan wordt om het juiste array-element te lezen of te schrijven.

3.0.2 Verschillende uitvoeringen

Voor beter besluiten te kunnen trekken hebben we ook een tijdsmeting uitgevoerd om zo te controleren hoelang telkens de flip functie duurt in verschillende configuraties.

3 blocks van 3 threads

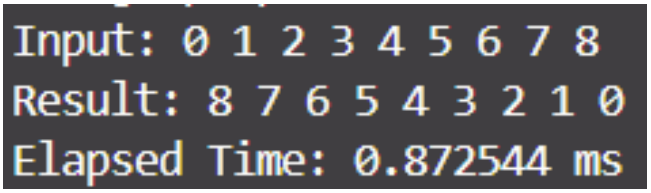


Figure 1: Output bij 3 blocks van 3 threads

1 block van 9 threads

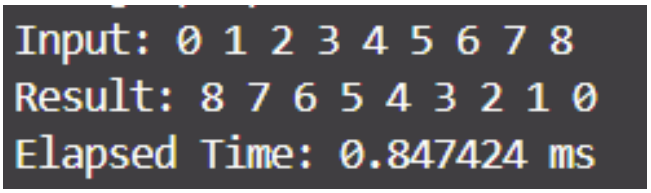


Figure 2: Output bij 1 block van 9 threads

9 blocks van 1 thread

Bij 9 blocks van 1 thread zien we een verbetering op vlak van timing. Deze is meestal sneller dan de andere configuraties als we de code meerdere keren uitvoeren.

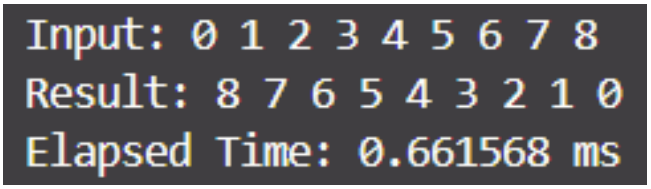


Figure 3: Output bij 9 blocks van 1 thread

1 block van 1 thread

Als we maar gebruik maken van 1 thread en 1 block dan zien we dat enkel 1 getalwaarde wordt uitgevoerd. Dit is ook wat we verwachten aangezien elke thread verantwoordelijk is voor één element in de array. De waarde N moet altijd groter of gelijk aan threadsxblocks zijn voor correcte werking.

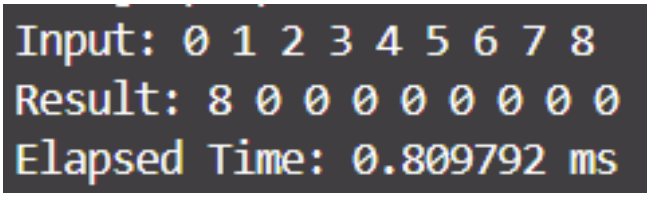


Figure 4: Output bij 1 block van 1 thread

4 Conclusie

In dit labo hebben we een eerste praktische kennismaking gekregen met CUDA en GPU-parallelisatie door een array om te keren met een kernel. We hebben gezien hoe threads en blocks samen de data verwerken en dat elke thread één element behandelt. Door verschillende configuraties van blocks en threads te testen, werd duidelijk dat meerdere kleine blocks vaak iets betere timings geven dan één groot block, al blijven de verschillen bij zo'n kleine array beperkt. Ook bleek dat het totaal aantal threads minstens gelijk moet zijn aan het aantal elementen om een volledig correct resultaat te krijgen. Deze oefening toont het basisprincipe van parallel uitvoeren van taken.